FINAL RELEASE AUDIT

# OpenJarvis v1.0

## Audit and Debug Checklist

This report presents the comprehensive pre-release audit of the OpenJarvis AI agent system, covering build state verification, cold-start testing, security audit, per-phase re-verification, scope-drift analysis, and documentation accuracy checks against the actual running application.

Audit Date: 2026-08-18
Auditor: Automated Audit System
Target: OpenJarvis Phases 0-12

# 1. Executive Summary

This audit was conducted against the actual running OpenJarvis application, not against code or self-reported claims. The project is a multi-phase, single-user, self-hosted AI agent system built with TypeScript, Express, Prisma (SQLite), and a Next.js 16 frontend. The BUILD_STATE.md claims completion through Phase 10 with 258 tests passing. The audit reveals a significant gap between claimed state and reality: while all 289 unit tests pass, three entire phases (9, 11, 12) are missing, the database migration history is incomplete, critical security vulnerabilities exist, and there is zero cold-start documentation for new users.

The most critical findings are: (1) no README, INSTALLATION, or QUICKSTART documentation exists, making cold-start impossible for a new user; (2) three phases are entirely missing with no acknowledgment in BUILD_STATE.md; (3) a dual permission system conflict between Phase 4 and Phase 10 causes tools to be permanently blocked; (4) all API routes lack authentication, exposing the agent execution and approval system to untrusted access; and (5) the .env file is tracked in git. These issues collectively prevent a v1.0 release.

---

### Audit Scorecard

| Audit Section | Status | Critical Issues |
|---|---|---|
| BUILD_STATE.md Accuracy | FAIL | 3 |
| Cold-Start Test | FAIL | 3 |
| Secrets & Credentials | FAIL | 5 |
| Per-Phase Verification | FAIL | 4 |
| Chaos / Resilience Tests | SKIP | N/A |
| Scope-Drift Check | FAIL | 3 |
| Documentation Accuracy | FAIL | 6 |
| Legal / Compliance | SKIP | N/A |
| Cost / Budget Sanity | WARN | 1 |

# 2. BUILD_STATE.md Reality Check

## 2.1 Current Phase Mismatch

BUILD_STATE.md declares "Phase 10 - Approval Workflow and Human-in-the-Loop (COMPLETED)" as the current phase. However, the document makes no mention of Phase 9 (Opportunity Engine), Phase 11 (API/SDK), or Phase 12 (Hardening). These phases are not listed as completed, pending, or deferred. The "Known Failures / Blockers" section also fails to acknowledge their absence. For a release audit, the BUILD_STATE.md must honestly reflect the true state of the project, including phases that were skipped or remain unimplemented. The Architecture Decisions Log contains entries through Phase 10 but has no notation indicating phases 9, 11, or 12 were deliberately skipped, which undermines the document as a reliable

source of truth.

### 2.2 File Structure Stale at Phase 7

The "File Structure" section in BUILD_STATE.md (lines 255-365) was frozen after Phase 7. It does not list any files from Phase 8 (MCP/Plugins), Phase 10 (Approvals/Authorization), or the mobile app shell. Specifically missing: all MCP files under src/mcp/ (mcpClient.ts, types.ts, transports.ts, pluginManager.ts), the approval system (approvalGate.ts, approvalService.ts, capabilityRegistry.ts), approval route and UI component, test files for phases 8 and 10, and the entire mini-services/openjarvis-mobile/ directory with approximately 30+ React Native files. The test count is also wrong: claimed 258, actual 289 (31 extra from phase10-auth-model.test.ts).

## 3. Cold-Start Test

The cold-start test simulates what a new developer would experience when cloning the repository and attempting to run the application for the first time. This is the single most important test in the audit, because if a stranger cannot get the app running by following the provided documentation, nothing else matters.

### 3.1 Documentation Availability

The audit found no README.md, INSTALLATION.md, or QUICKSTART.md at the project root. The only documentation is BUILD_STATE.md, a build tracker, not a setup guide. A new developer would have no instructions for installing dependencies, setting up the database, or starting the server. The .env.example file referenced in the Phase 0 acceptance checklist (".env.example exists and matches what the code actually reads") does not exist anywhere in the repository, making this checklist item demonstrably false. This is a complete cold-start failure.

### 3.2 Database Migration Gap

The Prisma migrations directory contains only 3 migration files: phase6_memory_enhancement, phase7_mobile_clients, and phase8_mcp_plugins. The base schema migrations covering the foundational tables (missions, mission_events, tools, memory_entries) from Phase 1 are missing. When prisma db push --force-reset is run, it syncs the full schema correctly, but prisma migrate status falsely reports "Database schema is up to date" because it only compares against the 3 known migrations. The start.sh script does not include a prisma migrate or prisma db push step, further compounding this issue. Without manual intervention, a fresh clone cannot set up the database correctly.

### 3.3 Server Boot Result

Despite documentation and migration issues, the server boots successfully when dependencies are installed and the database schema is synced manually. The Express API starts on port 3001, the WebSocket server starts on port 3002, and the /health endpoint returns a healthy status with real database connectivity (latency: 5-8ms). The server handles SIGTERM gracefully. However, reaching this point requires knowledge that is not documented anywhere in the repository.

# 4. Secrets and Credentials Audit

A comprehensive scan of the entire repository for hardcoded API keys, tokens, connection strings, and passwords found no secrets embedded in source code. All sensitive configuration correctly uses process.env references. However, several critical security issues were identified in how secrets are managed and how authentication is implemented.

| # | Severity | Issue | Location |
|---|---|---|---|
| 1 | CRITICAL | .env tracked in git despite .gitignore rule. Was committed before the rule was added, so git continues tracking it. | .env, .gitignore |
| 2 | CRITICAL | SQLite .db files tracked in git. dev.db contains MobileClient table with plaintext API keys. | prisma/dev.db |
| 3 | CRITICAL | Mobile API keys stored in plaintext (not hashed). No hashing library in dependencies. | schema.prisma:107 |
| 4 | CRITICAL | All core API routes have zero authentication. /agent/run can trigger AI execution consuming paid API credits. | index.ts |
| 5 | CRITICAL | Admin routes (/mobile/admin/*) have no authentication. Anyone can regenerate API keys. | mobileAdmin.ts |
| 6 | HIGH | WebSocket server has no auth and uses CORS origin: *. Any client can subscribe to events. | index.ts:58-68 |
| 7 | MEDIUM | No .env.example file exists. Required env vars are undocumented. | Project root |
| 8 | MEDIUM | No rate limiting on mobile registration endpoint. | routes/mobile.ts |

The positive finding is that no API keys or tokens are reachable from browser devtools. The frontend API client sends only Content-Type headers with no authentication tokens. However, the complete absence of authentication on all API routes is a critical security gap that must be addressed before any public release, even for a single-user self-hosted system.

# 5. Per-Phase Re-Verification

Each phase was re-verified against the actual running application, not trusted from the original session. All 289 unit tests pass when the database schema is properly synced. Live API endpoint tests confirm core functionality is working. However, several structural issues were found that cause runtime failures in specific scenarios.

## 5.1 Unit Test Results

| Test Suite | Tests | Pass | Fail | Verdict |
|---|---|---|---|---|
| Phase 1 - Foundation | 23 | 23 | 0 | ALL PASS |
| Phase 2 - Agent Runtime | 23 | 23 | 0 | ALL PASS |
| Phase 4 - Computer Control | 23 | 23 | 0 | ALL PASS |
| Phase 5 - Voice | 41 | 41 | 0 | ALL PASS |

| Test Suite | Tests | Pass | Fail | Verdict |
|---|---|---|---|---|
| Phase 6 - Memory | 46 | 46 | 0 | ALL PASS |
| Phase 7 - Mobile | 22 | 22 | 0 | ALL PASS |
| Phase 8 - MCP/Plugins | 36 | 36 | 0 | ALL PASS |
| Phase 10 - Approvals | 44 | 44 | 0 | ALL PASS |
| Phase 10 - Auth Model | 31 | 31 | 0 | ALL PASS |
| **TOTAL** | **289** | **289** | **0** | 100% PASS |

## 5.2 Critical Phase Issues

### Dual Permission System Conflict (Phase 4 vs Phase 10)

The most significant per-phase finding is a dual permission system conflict. Phase 4 introduced an in-memory PermissionManager with a binary granted/not-granted model. Phase 10 introduced a DB-backed CapabilityRegistry implementing the Authorization Model spec with three states (allowed/denied/undefined). The agent loop calls checkApprovalGate() which uses the DB-backed system. However, all 17 computer-control tool files also call getPermissionManager().check() inside their execute() method. This means a capability granted in the DB will be approved by the agent loop but then immediately refused by the tool itself, because the two systems do not share state. Granting a capability in the database does not grant it in the in-memory PermissionManager.

### filesystem_delete Permanently Broken

The filesystem_delete tool contains a hardcoded block at lines 166-172 of filesystem.ts that always returns a requires_approval error with the message "Filesystem delete requires human approval via the approval queue (Phase 9)". This code was written as a placeholder during Phase 4, anticipating that Phase 9 would provide the approval queue. Now that Phase 10 has implemented the approval system, this hardcoded block was never removed. The result is that filesystem_delete is permanently non-functional regardless of permission grants or approvals.

### Missing Phases: 9, 11, 12

Phase 9 (Opportunity Engine / Approvals for spending), Phase 11 (API/SDK with rate limiting), and Phase 12 (Hardening with chaos tests) have zero implementation in the codebase. No test files, no source code, no route definitions, and no acknowledgment in BUILD_STATE.md. The opportunity engine concept (spending scoring, discovery sources, approval-for-spend) is entirely absent. No SDK client library, no rate limiting middleware, no chaos test suite, no circuit breakers, and no graceful degradation code exists anywhere in the repository.

# 6. Chaos and Resilience Tests

The checklist requires six specific chaos tests: DB disconnection mid-mission, backend process kill mid-tool-call, network disconnect mid-voice-session, malformed tool response, worker

restart mid-mission, and duplicate event processing. None of these tests have been implemented. The entire Phase 12 (Hardening) is absent from the codebase. There are no circuit breakers, no graceful degradation mechanisms, no retry logic for database connections, and no duplicate detection for webhooks or messages. The agent loop is synchronous and will crash if the database connection is lost mid-execution. Mission state is not checkpointed, so a process kill would lose the current execution context. WebSocket connections do not have reconnection logic with state recovery. All six chaos tests are marked as SKIP.

# 7. Scope-Drift Check

## 7.1 Memory Operations Completeness

The memory system (Phase 6) implements most required operations at the database level via Prisma. Create, read, search, update, delete, recall, forget, associations, consolidation, and purge are all fully functional and verified through 46 passing tests. However, two operations are missing: "export" (bulk download of memory entries in a structured format) and "disable/deactivate" (temporarily disabling a memory entry without deleting it). Neither has a service method, route endpoint, or UI control.

## 7.2 Approval-Required Actions at Tool Layer

The approval gate (Phase 10) is called by the agent loop before every tool execution, which correctly enforces approval requirements at the execution layer. However, the dual permission system issue (Section 5.2) means that computer-control tools have a second, independent permission check inside their execute() method that uses the Phase 4 in-memory system. This creates a gap where the approval gate might approve an action but the tool itself refuses it. The approval system is architecturally correct but operationally broken for 17 tools.

## 7.3 UI vs. Backend Consistency

The dashboard has 5 tabs (Missions, Tools, Memory, Settings, Approvals) that correspond to real backend endpoints. No fake or placeholder components were found. However, the capability grant management system has full backend CRUD endpoints and corresponding frontend API client functions in openjarvis-api.ts, but no UI component exposes these capabilities to the admin. This means the Authorization Model ("The admin is the policy") cannot be managed through the UI, a significant gap for the stated design goal.

## 7.4 Structured Error Format Consistency

The structured error format {error: {code, message, requestId}} is used correctly across most routes via the centralized error handler middleware. All routes that throw AppError get properly formatted responses. The exception is voice.ts, which uses inline res.status().json({error: {...}}) at 14 separate error points instead of throwing AppError. While the output format is identical, these inline responses bypass the error handler, meaning errors are not logged via logger.error(), the isOperational flag is not set, and stack traces are not captured

for debugging.

## 8. Documentation Accuracy

Documentation drift is severe. The audit checked for README, QUICKSTART, INSTALLATION, ARCHITECTURE, CONFIGURATION, SECURITY, TROUBLESHOOTING, and API documentation. None exist as standalone documents. The only documentation is BUILD_STATE.md, which serves as a combined build tracker, architecture log, and file structure reference, but it is stale and incomplete.

| Document | Exists | Accurate | Issue |
|---|---|---|---|
| README.md | NO | N/A | No project README |
| QUICKSTART.md | NO | N/A | No quickstart guide |
| INSTALLATION.md | NO | N/A | No installation guide |
| ARCHITECTURE.md | NO | N/A | Only in BUILD_STATE.md (stale) |
| CONFIGURATION.md | NO | N/A | No env var documentation |
| SECURITY.md | NO | N/A | No security documentation |
| TROUBLESHOOTING.md | NO | N/A | No troubleshooting guide |
| API docs | NO | N/A | No OpenAPI/Swagger spec |
| LICENSE | NO | N/A | No project-root LICENSE file |
| .env.example | NO | N/A | Claimed in Phase 0 but missing |

Environment variables actually used by the code: DATABASE_URL, PORT, WS_PORT, GEMINI_API_KEY, GROQ_API_KEY, VOICE_PROVIDER, APPROVAL_TTL_SECONDS, LOG_LEVEL. BUILD_STATE.md additionally mentions VOICE_LANGUAGE, VOICE_TTS_VOICE, and VOICE_MAX_AUDIO_SIZE, but these were not found in the actual source code, suggesting they were planned but never implemented.

## 9. Legal and Compliance Spot-Check

The legal and compliance checks are largely not applicable because the critical phases containing the relevant features have not been implemented. Telephony (Phase 10 in the checklist) does not exist: no Twilio integration, no consent management, no do-not-call list checking, no AI-disclosure logic. The opportunity engine (Phase 9) that would handle discovery source compliance has zero implementation. The only compliance-relevant finding is the absence of a project-root LICENSE file. BUILD_STATE.md mentions MIT license per the phase spec, but only the mobile sub-project has a license file. All other legal checks are marked as SKIP.

## 10. Cost and Budget Sanity Check

The budget cap system (Phase 2) is implemented and tested: missions track token usage and tool call counts, and the agent loop halts to "blocked" status when either limit is exceeded. However, a full end-to-end test with real API calls cannot be performed because no GEMINI_API_KEY or GROQ_API_KEY is available in this environment. The approval system (Phase 10) adds a secondary spending control layer by requiring human approval for certain tool executions, but the dual permission system conflict means this is not functioning correctly for computer-control tools. There is no mechanism to track cumulative spending across missions or enforce a global spending limit.

## 11. Open Issues Log

The following table tracks all issues discovered during this audit. No v1.0 release should be tagged until this log is empty or every remaining item is explicitly deferred with documented reasoning.

| # | Section | Issue | Severity | Status |
|---|---|---|---|---|
| 1 | 2.1 | Phase 9, 11, 12 missing from BUILD_STATE.md | HIGH | OPEN |
| 2 | 2.2 | File Structure section frozen at Phase 7 | MEDIUM | OPEN |
| 3 | 2.3 | Test count claimed 258, actual 289 | LOW | OPEN |
| 4 | 3.1 | No README, INSTALLATION, or QUICKSTART docs | CRITICAL | OPEN |
| 5 | 3.1 | .env.example does not exist (claimed in Phase 0) | CRITICAL | OPEN |
| 6 | 3.2 | Base schema migrations missing (Phase 1-4 tables) | HIGH | OPEN |
| 7 | 3.2 | start.sh does not run prisma migrate/db push | MEDIUM | OPEN |
| 8 | 4 | .env tracked in git | CRITICAL | OPEN |
| 9 | 4 | SQLite .db files tracked in git (plaintext API keys) | CRITICAL | OPEN |
| 10 | 4 | Mobile API keys stored in plaintext, not hashed | CRITICAL | OPEN |
| 11 | 4 | All core API routes have zero authentication | CRITICAL | OPEN |
| 12 | 4 | Admin routes have zero authentication | CRITICAL | OPEN |
| 13 | 4 | WebSocket has no auth, CORS allows all origins | HIGH | OPEN |
| 14 | 4 | No rate limiting on mobile registration | MEDIUM | OPEN |
| 15 | 5.2 | Dual permission systems: Phase 4 vs Phase 10 | CRITICAL | OPEN |
| 16 | 5.2 | filesystem_delete permanently broken (Phase 9 block) | CRITICAL | OPEN |
| 17 | 5.2 | Phase 9 (Opportunity Engine) not implemented | HIGH | OPEN |
| 18 | 5.2 | Phase 11 (API/SDK) not implemented | HIGH | OPEN |
| 19 | 5.2 | Phase 12 (Hardening) not implemented | HIGH | OPEN |
| 20 | 6 | No chaos/resilience tests exist | HIGH | OPEN |
| 21 | 7.1 | Memory export and disable operations missing | LOW | OPEN |

| # | Section | Issue | Severity | Status |
|---|---|---|---|---|
| 22 | 7.3 | Capability grant management has no UI | MEDIUM | OPEN |
| 23 | 7.4 | Voice routes bypass error handler (14 points) | MEDIUM | OPEN |
| 24 | 8 | No project-root LICENSE file | MEDIUM | OPEN |
| 25 | 8 | 3 env vars in BUILD_STATE not in code | LOW | OPEN |
| 26 | 9 | Telephony, DNC, AI-disclosure not implemented | HIGH | OPEN |
| 27 | 10 | No E2E budget verification (no API keys) | MEDIUM | OPEN |

# 12. Final Verdict

Based on the comprehensive audit presented in this report, the OpenJarvis project is **NOT ready for v1.0 release**. While the engineering quality of the implemented phases is solid (289/289 tests pass, clean architecture, proper separation of concerns), the project has critical gaps in documentation, security, and completeness that collectively prevent a release tag.

The three most impactful blocker categories are: (1) the complete absence of onboarding documentation, which means no new user can get the application running; (2) the authentication gap on all API routes, which is unacceptable even for a self-hosted single-user system because it exposes the agent execution pipeline to any network-adjacent attacker; and (3) the dual permission system conflict, which makes 17 computer-control tools non-functional despite both permission systems individually passing their tests.

The missing phases (9, 11, 12) represent significant feature gaps but could potentially be deferred with explicit documentation if the admin decides they are not required for the initial release. However, the BUILD_STATE.md must be updated to acknowledge these deferrals with documented rationale before any release is considered.

**VERDICT: NOT READY FOR RELEASE**

27 open issues (7 Critical, 8 High, 8 Medium, 4 Low)
3 missing phases (9, 11, 12) unacknowledged
0 cold-start documentation files
0 authenticated API routes

Recommended: Address all CRITICAL issues, create minimum documentation (README + .env.example), resolve the dual permission system, then re-run this audit.